

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-adec01c3acae53179.jsonl`  
- SHA-256: `f31c49593fc18aa009804b744fe65806f3d860c51c0564d5694d2ed20a1984dd`  
- Source modified: `2026-06-13T19:15:31+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.209Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title": "family='fusion' windowing threshold and substrate runner config read from different config sections ? silent A/B drift", "severity": "med", "file": "backend/pipeline/segmenters/fusion_hybrid.py:58-68 (and trajectory_hybrid.py:149)", "detail": "FusionSegmenter sets family='fusion', so the engine reads the WINDOWING velocity threshold from idx_cfg['fusion']['velocity_threshold'] (trajectory_hybrid.py:149: idx_cfg.get(self.family, {}).get('velocity_threshold', 0.02)). But _build_runner builds the substrate via WasbConfig.from_indexing_cfg(idx_cfg) / TrackNetConfig.from_indexing_cfg(idx_cfg), which read idx_cfg['wasb'] / idx_cfg['tracknet'] (wasb_runner.py:46: (idx_cfg or {}).get('wasb', {})), NOT idx_cfg['fusion']. So weights/conda/WSL come from the wasb section while the windowing threshold comes from the fusion section. Consequence: a user who has tuned indexing.wasb.velocity_threshold and runs fusion_hybrid silently gets the DEFAULT 0.02 windowing from fusion.velocity_threshold, producing a DIFFERENT candidate-window set than the wasb_hybrid run it is meant to A/B against. The module docstring (fusion_hybrid.py:20-22) claims 'Defaults reproduce the substrate exactly' and seeds 'identically to wasb_hybrid' ? that only holds when indexing.wasb.velocity_threshold is also at its default 0.02. The FusionConfig comment (models.py) hints at this but the equivalence-with-tuned-wasb claim is a trap.", "fix": "Either make FusionConfig.velocity_threshold default to None and fall back to the substrate's own velocity_threshold (idx_cfg['wasb']/'tracknet') when unset, or have the engine prefer the substrate section for the fusion family. At minimum, tighten the docstring so it doesn't promise identical seeding against a tuned wasb run without mirroring velocity_threshold into the fusion section."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:55.917Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:57.108Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9        GPU-free, computed from the trajectory we already have); and, only when the person cue
10       is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11       (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12       fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13       never imports torch / touches the GPU.
14       - ``_select_for_handoff`` ? optional served Gate A/B: when
15       ``indexing.fusion.min_crossings``
16       (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17       are
18       dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19       **Persisted candidates remain the full superset** regardless, so the offline
20       experiment
21       harness can re-score any variant.
22
23       Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24       but
25       nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26       seeds
27       and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28       default
29       segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30       through the
31       experiment harness (EXPERIMENT_HARNESS.md), never silently.
32       """
33       from typing import Any, Dict, List, Optional, Tuple
34
35       from backend.pipeline.segmenters.base import segmenter_registry
36       from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37       from backend.pipeline.detectors.base import DetectorRunner
38       from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39       from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40       from backend.pipeline.detectors import rally_gate
41       from backend.pipeline.detectors import fusion_features as ff
42
43       class FusionSegmenter(TrajectoryHybridSegmenter):
44           """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45           B)."""
46
47           family = "fusion"
48           display_label = "Fusion"
49
50           def __init__(self, db: Any, config: Any):
51               super().__init__(db, config)
52               # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53               default path).
54               self._person: Optional[Any] = None
55
56           @property
57           def name(self) -> str:
58               return "fusion_hybrid"
59
60           @property
61           def description(self) -> str:
62               return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
```

```

rallies, "
55         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56         "AI provider sets semantic boundaries.")
57
58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                frame_width: float, video_path: str,
72                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                          frame_width: float, video_path: str,
87                          fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,

```

```

107         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                             window_stats: List[Dict[str, Any]],
111                             idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:13:57.520Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35

```

```

36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84         -----
85         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86         frame_width: float, video_path: str,
87         idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89             BASE
90             returns exactly the 4 motion keys, so the trajectory twins persist byte-
91             identical
92             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
93             features). ``points`` are the whole-video trajectory points
94             (TrajectoryPoint)."""
95             return {
96                 "peak_velocity": cw["peak_velocity"],
97                 "mean_velocity": cw["mean_velocity"],
98                 "active_frame_count": cw["active_frame_count"],
99                 "track_id_count": cw["track_id_count"],
100             }

```

```

95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                             window_stats: List[Dict[str, Any]],
98                             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                      player_pool: Optional[List[str]] = None,
108                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)

```

```

156
157     detection_params = {
158         "detector": self.name,
159         "velocity_thresh": velocity_thresh,
160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184                 len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                 label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label} rally]
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198 f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200 f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     fusion
205     # segmenter adds net_crossings + person-cue features. Computed once, reused for
206     both
207     # persistence and the handoff gate below.
208     window_stats = [
209         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210         idx_cfg)
211         for cw in all_candidate_windows
212     ]
213
214     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216     if store_candidates:
217         for i, cw in enumerate(all_candidate_windows):
218             candidate_ids[i] = self.db.add_candidate_segment(
219                 video_id, detector=self.name,
220                 start_time=cw["start"], end_time=cw["end"],

```

```

215             chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"])),
216             stats=window_stats[i], detection_params=detection_params,
217         )
218
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224     # candidates above stay the full superset ? only the served play_segments are
gated.
225     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227     os.makedirs(handoff_dir, exist_ok=True)
228     logger.info("Phase 3: %s validation on %d candidate windows...",
229               provider_name, len(handoff_indices))
230
231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                           label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["_candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                   for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:

```



```

271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start
275             metadata = {
276                 "shot_count": self._shot_count_from(segment, dur),
277                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                 "detector": f"{self.family}-hybrid-{model_name}",
279             }
280             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281             if segment_id:
282                 candidate_id = segment.get("_candidate_id")
283                 if candidate_id is not None:
284                     self.db.link_candidate_to_segment(candidate_id, segment_id)
285                 segments_data.append({
286                     "id": segment_id, "start_time": start, "end_time": end,
287                     "duration": dur, "metadata": metadata,
288                 })
289             except Exception as e:
290                 logger.warning("Skipping invalid segment struct: %s", e)
291
292         return segments_data
293

```

5. assistant (2026-06-13T19:14:05.650Z)

Now let me look at the config models and the runner config sources to verify the finding's claims about config sections.

6. user (2026-06-13T19:14:07.120Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30    @dataclass

```

```

31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43
44     @classmethod
45     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46         w = (idx_cfg or {}).get("wasb", {}) or {}
47         return cls(
48             distro=w.get("wsl_distro", "Ubuntu"),
49             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51             conda_env=w.get("conda_env", "wasb"),
52             weights_path=w.get("weights_path",
53                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54             sport=w.get("sport", "badminton"),
55             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56             timeout_sec=int(w.get("timeout_sec", 1800)),
57             keep_frames=bool(w.get("keep_frames", False)),
58             min_free_gb=float(w.get("min_free_gb", 0.0)),
59         )
60
61
62     class WasbRunner(WslCommandMixin, DetectorRunner):
63         def __init__(self, cfg: WasbConfig):
64             self.cfg = cfg
65
66         def healthcheck(self) -> Tuple[bool, str]:
67             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68             cmd = (f"conda activate {self.cfg.conda_env} && "
69                   f"python -c \"import torch; print('torch', torch.__version__, "
70                   f"'cuda', torch.cuda.is_available())\"")
71             try:
72                 res = self._wsl_bash(cmd)
73             except FileNotFoundError:
74                 return False, "`wsl` not found ? Windows + WSL2 required."
75             except subprocess.TimeoutExpired:
76                 return False, "WSL healthcheck timed out."
77             out = (res.stdout or "").strip()
78             if res.returncode != 0:
79                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
80             return True, out
81
82         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
83             src_mnt = to_wsl_mnt_path(video_win_path)
84             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
85             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
86             try:
87                 res = self._wsl_bash(cmd)
88             except subprocess.TimeoutExpired:
89                 logger.error("Staging video into WSL timed out.")

```

```
90         return None
```

7. user (2026-06-13T19:14:07.530Z)

```
1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
```

```

61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66     sport: str = "badminton"
67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73     keep_frames: bool = False
74     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75     min_free_gb: float = 0.0
76
77
78     class FusionConfig(BaseModel):
79         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81         Every default reproduces control behaviour: the fusion segmenter persists the
82         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83         true ? the person-cue features, but it does NOT gate the served handoff unless a
84         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86         supplied ? off-court persons (spectators / adjacent courts) are excluded.
87         """
88         # Which trajectory substrate seeds the windows (shuttle stays primary).
89         substrate: Literal["wasb", "tracknet"] = "wasb"
90         # Windowing velocity threshold for the fusion family (the engine reads
91         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93         velocity_threshold: float = 0.02
94         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95         # enabled ? so the default path never imports torch / touches the GPU.
96         cue_flags: dict[str, bool] = Field(default_factory=dict)
97         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99         min_crossings: int = Field(default=0, ge=0)
100         # Served Gate B: when the person cue is on, drop windows with median in-court
players
101         # < this. 0 = OFF.
102         min_players: int = Field(default=0, ge=0)
103         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104         court_polygon: Optional[list[list[float]]] = None
105         # Net line for the person two-sided-motion split (person features only ? the shuttle
106         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107         net_axis: Literal["x", "y"] = "y"
108         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111     class IndexingConfig(BaseModel):
112         default_segmenter: str = "yolo_hybrid"
113         use_gpu: bool = True
114         motion_threshold: float = 0.08

```

```

115 min_segment_duration: float = 3.0
116 dead_time_merge_gap: float = 2.0
117 chunk_duration: float = 90.0
118 chunk_overlap: float = 10.0
119 detector_model: str = "fasterrcnn_resnet50_fpn"
120 detector_score_threshold: float = 0.5
121 yolo_velocity_threshold: float = 0.15
122 yolo_frame_skip: int = 3
123 yolo_tracker: str = "bytetrack.yaml"
124 yolo_prefetch_workers: int = 2
125 min_action_window_duration: float = 2.0
126 log_yolo_candidates: bool = True
127 store_yolo_candidates: bool = True
128 ai_handoff_padding: float = 2.0
129 ai_max_workers: int = 5
130 # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
131 # chunks of a high-bitrate (4K) source blow past the provider upload cap
132 # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
133 # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
134 # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
135 # Matches gemini_refine's --clip-scale-width default.
136 ai_chunk_scale_width: int = 1280
137 # Optional debug knob: trim every video to the first N seconds before processing.
138 debug_duration: Optional[float] = None
139
140 tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
141 wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
142 fusion: FusionConfig = Field(default_factory=FusionConfig)
143
144 @field_validator("default_segmenter")
145 @classmethod
146 def segmenter_must_be_registered(cls, v: str) -> str:
147     # Registry check happens at runtime in main/segmenter selection.
148     # Here we just allow any string for forward compatibility.
149     return v
150
151 @model_validator(mode="after")
152 def _chunk_step_must_be_positive(self) -> "IndexingConfig":
153     # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
154     # step makes `while t < duration: t += step` loop forever, growing memory before
any
155     # GPU work. Reject the bad config at load time instead.
156     if self.chunk_overlap >= self.chunk_duration:
157         raise ValueError(
158             f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
159             f"({self.chunk_duration}); otherwise chunking never advances."
160         )
161     return self
162
163
164 class OutputConfig(BaseModel):
165     default_highlights_name: str = "highlights.mp4"
166     db_name: str = "sports_indexer.db"
167     # Directory where the DB, highlight reels, and processed clips are written.
168     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
169     output_dir: str = "output"
170
171
172 class WatermarkConfig(BaseModel):
173     """Branding overlay burned into each highlight clip during the per-clip re-encode
174     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
175     (under `stitching.watermark` in config.json) to turn it off. The text is fully
176     configurable. The concat stage stays stream-copy (-c copy)."""

```

```

177     enabled: bool = True
178     text: str = "Generated by Khelsutra.guru"
179     logo_path: str = ""          # optional transparent PNG overlay; "" = no logo
180     font_path: str = ""         # optional .ttf; "" = use the fontconfig `font` family
below
181     font: str = "Sans"          # fontconfig family used when font_path is empty
182     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
183     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
184     box: bool = True            # faint backing box behind the text for legibility
185     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
186     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
187
188
189     class StitchingConfig(BaseModel):
190         begin_padding: float = 0.5
191         end_padding: float = 0.5
192         use_cache: bool = False
193         min_shot_count: int = 1
194         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
195
196
197     class SecurityConfig(BaseModel):
198         # When set, /api/process video_path must resolve under this directory (hosted/tenant
199         # mode). Default None preserves the single-user local 'any local file' workflow.
200         allowed_video_root: Optional[str] = None
201
202         # --- Chunked upload (B2, PR-2) ---
203         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
204         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
205         # contain upload_dir or /api/process will reject the uploaded paths.
206         upload_dir: str = "output/uploads"
207         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
208         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
209         max_upload_mb: int = 4096
210         max_part_mb: int = 95
211         allowed_upload_extensions: List[str] = ["mp4", "mov"]
212
213
214     class AppConfig(BaseModel):
215         """Root configuration object.
216
217         All runtime configuration should be accessed via an instance of this class
218         (or the loader) rather than raw dict lookups.
219         """
220
221         sport: Sport = "badminton"
222         ai_provider: str = "gemini"
223         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
224         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
225         ai_model: str = "gemini-flash-latest"
226
227         validation: ValidationConfig = Field(default_factory=ValidationConfig)
228         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
229         output: OutputConfig = Field(default_factory=OutputConfig)
230         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
231         security: SecurityConfig = Field(default_factory=SecurityConfig)
232
233         # Allow extra keys during transition so existing config.json files do not break.
234         model_config = {
235             "extra": "allow",
236             "validate_assignment": True,

```

```

237     }
238
239     # Temporary migration shim for legacy dict-style access (config.get("section",
240     {}).get(...)).
241     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
242     # attribute access / the typed config. A nested *section* is returned as a plain
243     dict
244     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
245     # section model instead breaks it (a model has no .get), which bit every validator
246     on
247     # real runs. Scalar leaves are returned as-is.
248     def get(self, key: str, default: Any = None) -> Any:
249         _MISSING = object()
250         val: Any = _MISSING
251         if key in type(self).model_fields or hasattr(self, key):
252             val = getattr(self, key, _MISSING)
253         if val is _MISSING:
254             # Legacy convenience: find a leaf key inside any nested section. Sections
255             # are
256             # discovered dynamically from the model fields, so a future top-level
257             section
258             # (e.g. `audio`, `jobs`) is covered automatically without editing a
259             hardcoded list.
260             for fname in type(self).model_fields:
261                 section = getattr(self, fname, None)
262                 if isinstance(section, BaseModel) and hasattr(section, key):
263                     val = getattr(section, key)
264                     break
265             if val is _MISSING:
266                 return default
267             if isinstance(val, BaseModel):
268                 return val.model_dump(mode="json")
269             return val
270
271     def __getitem__(self, key: str) -> Any:
272         val = self.get(key)
273         if val is None and key not in type(self).model_fields:
274             # Re-raise like a real dict for unknown top-level
275             raise KeyError(key)
276         return val
277
278     # Convenience type alias for callers that want to annotate
279     Config = AppConfig
280
281

```

8. assistant (2026-06-13T19:14:16.057Z)

The finding's claims are now verifiable against the code. Let me confirm the key facts:

1. ****Line 149 of trajectory_hybrid.py****: `velocity\_thresh = idx\_cfg.get(self.family, {}).get("velocity\_threshold", 0.02)` ? for `FusionSegmenter`, `self.family = "fusion"`, so it reads from `idx\_cfg["fusion"]["velocity\_threshold"]`. ? Confirmed.
2. ****Line 67-68 of fusion_hybrid.py****: `WasbConfig.from\_indexing\_cfg(idx\_cfg)` reads `idx\_cfg["wasb"]` (wasb_runner.py:46). ? Confirmed ? but note `WasbConfig` does NOT carry `velocity\_threshold` (it's not a field on the dataclass, lines 30-59), so the substrate config only contributes weights/conda/WSL, not the windowing threshold.
3. ****FusionConfig.velocity_threshold default = 0.02**** (models.py:93), and the docstring comment at models.py:90-92 explicitly acknowledges this: "set equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run."

Let me check whether `wasb\_hybrid` (the WasbHybridSegmenter) actually reads its velocity threshold from `indexing.wasb.velocity\_threshold` or somewhere else, since the A/B-drift claim

hinges on where the wasb run gets its threshold.

9. user (2026-06-13T19:14:17.152Z)

```
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\__init__.py
```

10. user (2026-06-13T19:14:18.396Z)

```
backend\pipeline\segmenters\trajectory_hybrid.py:47:    #: config section under `indexing`
(velocity_threshold lives there), the
backend\pipeline\segmenters\trajectory_hybrid.py:149:        velocity_thresh =
idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
backend\pipeline\segmenters\fusion_hybrid.py:40:        family = "fusion"
backend\pipeline\segmenters\tracknet_hybrid.py:22:        family = "tracknet"
backend\pipeline\segmenters\wasb_hybrid.py:24:        family = "wasb"
backend\pipeline\segmenters\yolo_hybrid.py:99:        velocity_thresh =
idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of frame width per second
```

11. assistant (2026-06-13T19:14:22.251Z)

This confirms the core mechanism. Let me read `wasb\_hybrid.py` to confirm `WasbHybridSegmenter` has `family = "wasb"` and thus reads its velocity threshold from `indexing.wasb.velocity\_threshold` (the same shared engine line 149).

12. user (2026-06-13T19:14:22.540Z)

```
1      """WASB-based hybrid play segmenter.
2
3      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5      badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9      min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
```



```

31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41     segmenter_registry.register(WasbHybridSegmenter)
42

```

13. assistant (2026-06-13T19:14:50.133Z)

This fully confirms the mechanism. Let me trace the A/B-drift scenario precisely:

```

**`wasb_hybrid` run** (family=`"wasb"`): line 149 reads `idx_cfg["wasb"]["velocity_threshold"]`.
So if a user tunes `indexing.wasb.velocity_threshold` to, say, `0.05`, the wasb run windows at
`0.05`.

```

```

**`fusion_hybrid` run** (family=`"fusion"`, substrate=wasb):
- line 149 reads `idx_cfg["fusion"]["velocity_threshold"]`, which defaults to `0.02`
(FusionConfig.velocity_threshold) and is NOT populated from the wasb section.
- `_build_runner` (line 67) builds `WasbConfig.from_indexing_cfg(idx_cfg)` reading
`idx_cfg["wasb"]` for weights/conda/WSL ? but `WasbConfig` has no `velocity_threshold` field, so
the tuned `indexing.wasb.velocity_threshold = 0.05` does NOT flow into the fusion windowing.

```

So the fusion run windows at `0.02` while the wasb run windows at `0.05` ? ****different candidate-window sets****. The finding's core claim holds.

Now let me adversarially test the finding's weak points:

1. The finding says "substrate runner config read from wasb section" implies the substrate's velocity_threshold is read from wasb ? but actually `WasbConfig` doesn't even read velocity_threshold at all. The windowing threshold for *both* wasb_hybrid and fusion_hybrid comes solely from line 149 (the `family` section), never from `WasbConfig`. The finding's fix suggestion "fall back to the substrate's own velocity_threshold (idx_cfg['wasb']/'tracknet'])" is slightly imprecise about *where* the substrate's threshold lives (it's `idx\_cfg['wasb']['velocity\_threshold']`, a config key, not a `WasbConfig` field), but the underlying drift it describes is real.

2. Is the docstring claim actually overstated? fusion_hybrid.py:20-24 says "Defaults reproduce the substrate exactly... seeds and hands off identically to `wasb\_hybrid`." This is true ****only** when wasb's velocity_threshold is also at default 0.02**. With a tuned wasb section, the two diverge. The docstring does not carry the caveat; the FusionConfig comment (models.py:90-92) DOES carry it. So the finding's nuance ("the FusionConfig comment hints at this but the equivalence-with-tuned-wasb claim is a trap") is accurate.

Let me verify the severity by checking whether there's a `dead\_time\_merge\_gap`/`min\_action\_window\_duration` per-family wrinkle too ? those are read from top-level `idx\_cfg` (lines 150-151), so they're shared and don't drift. Only `velocity\_threshold` is family-scoped. This is a real but narrow drift, gated behind a non-default config (a user must tune `indexing.wasb.velocity\_threshold`) and a non-default segmenter (fusion is opt-in). The consequence is silent A/B confound rather than a crash or data corruption.

Let me double-check there's no other place that mirrors `wasb.velocity\_threshold` into `fusion.velocity\_threshold` (e.g. config loader/migration) that would refute the drift.

14. user (2026-06-13T19:14:51.225Z)

```

backend\config\models.py:43:    velocity_threshold: float = 0.02
backend\config\models.py:69:    velocity_threshold: float = 0.02
backend\config\models.py:91:    # indexing.<family>.velocity_threshold). Default matches the
substrates' 0.02; set
backend\config\models.py:92:    # equal to indexing.wasb.velocity_threshold to A/B fusion
against a tuned wasb run.
backend\config\models.py:93:    velocity_threshold: float = 0.02
backend\config\models.py:121:    yolo_velocity_threshold: float = 0.15
backend\pipeline\segmenters\trajectory_hybrid.py:47:    #: config section under `indexing`
(velocity_threshold lives there), the
backend\pipeline\segmenters\trajectory_hybrid.py:149:        velocity_thresh =
idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
backend\pipeline\segmenters\yolo_hybrid.py:99:        velocity_thresh =
idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of frame width per second

```

15. assistant (2026-06-13T19:14:58.931Z)

No mirroring/migration logic exists that copies `wasb.velocity\_threshold` into `fusion.velocity\_threshold`. The drift is real and unmitigated. There are exactly three `velocity\_threshold` config sites (`tracknet`, `wasb`, `fusion`), each independent, and line 149 reads only the active family's section.

I have fully verified the finding. Let me record my conclusion.

16. user (2026-06-13T19:15:17.101Z)

Structured output provided successfully

17. assistant (2026-06-13T19:15:31.745Z)

Verified the finding against the actual code. It is **real but LOW severity** (the review rated it med).

****Confirmed mechanism:****

- `trajectory\_hybrid.py:149` reads the windowing threshold from `idx\_cfg[self.family]["velocity\_threshold"]`. For `FusionSegmenter`, `family = "fusion"` (`fusion\_hybrid.py:40`), so it reads `idx\_cfg["fusion"]["velocity\_threshold"]`, which defaults to `0.02` (`models.py:93`).
- `fusion\_hybrid.py:67-68` builds the substrate via `WasbConfig.from\_indexing\_cfg(idx\_cfg)` reading `idx\_cfg["wasb"]` (`wasb\_runner.py:46`) ? but `WasbConfig` has no `velocity\_threshold` field at all, so the tuned wasb threshold never flows into fusion windowing.
- `WasbHybridSegmenter` has `family = "wasb"` (`wasb\_hybrid.py:24`), so the wasb_hybrid run reads `idx\_cfg["wasb"]["velocity\_threshold"]` through the same line 149.
- No loader/migration mirrors wasb?fusion (Grep over `backend` shows three independent `velocity\_threshold` sites with no copy logic).

****Result:**** a user who tunes `indexing.wasb.velocity\_threshold` gets a wasb_hybrid run at the tuned value but a fusion_hybrid run still at `0.02` ? genuinely different candidate-window sets, silently confounding the A/B. The docstring at `fusion\_hybrid.py:20-24` overstates this ("seeds and hands off identically to wasb_hybrid") with no caveat; only the `FusionConfig` comment (`models.py:90-92`) carries the warning.

****Why LOW not med:**** no crash/data-loss/security impact; requires both the opt-in (non-default) fusion segmenter AND a manually-tuned non-default `indexing.wasb.velocity\_threshold`. Stock config makes the two paths identical. Only one parameter drifts ? `merge\_gap`/`min\_window\_duration`/`ai\_handoff\_padding` (`trajectory\_hybrid.py:150-152`) are top-level and shared. It's a sharp-edge/doc-overclaim in an experiment-harness context, not a production correctness bug.

****Minor imprecision**** in the finding (doesn't affect validity): it frames the substrate threshold as coming from the wasb section "via the runner," but `WasbConfig` doesn't read `velocity\_threshold` ? that key is consumed only by line 149 when `family=="wasb"`. The fix's `idx\_cfg['wasb']` fallback should target that config key, not a `WasbConfig` field.

Relevant files:

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory\_hybrid.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\fusion\_hybrid.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\wasb\_hybrid.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb\_runner.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py`